Lab Procedure for Python
Self-Localization

Setup

1. It is recommended that you review Lab 1 – Application Guide before starting this lab.

2. Launch Quanser Interactive Labs, scroll to the "QBot Platform" menu item and then select the "Warehouse" world.

3. Open and run the script localization.py. When the script is run successfully, it spawns the QBot and other elements within the virtual environment. Verify that the QBot is spawned as shown in Figure 1. Open the camera menu by pressing the 3 horizontal lines and set the Lock camera location to the "On" position. Press the "U" key and stop the script.

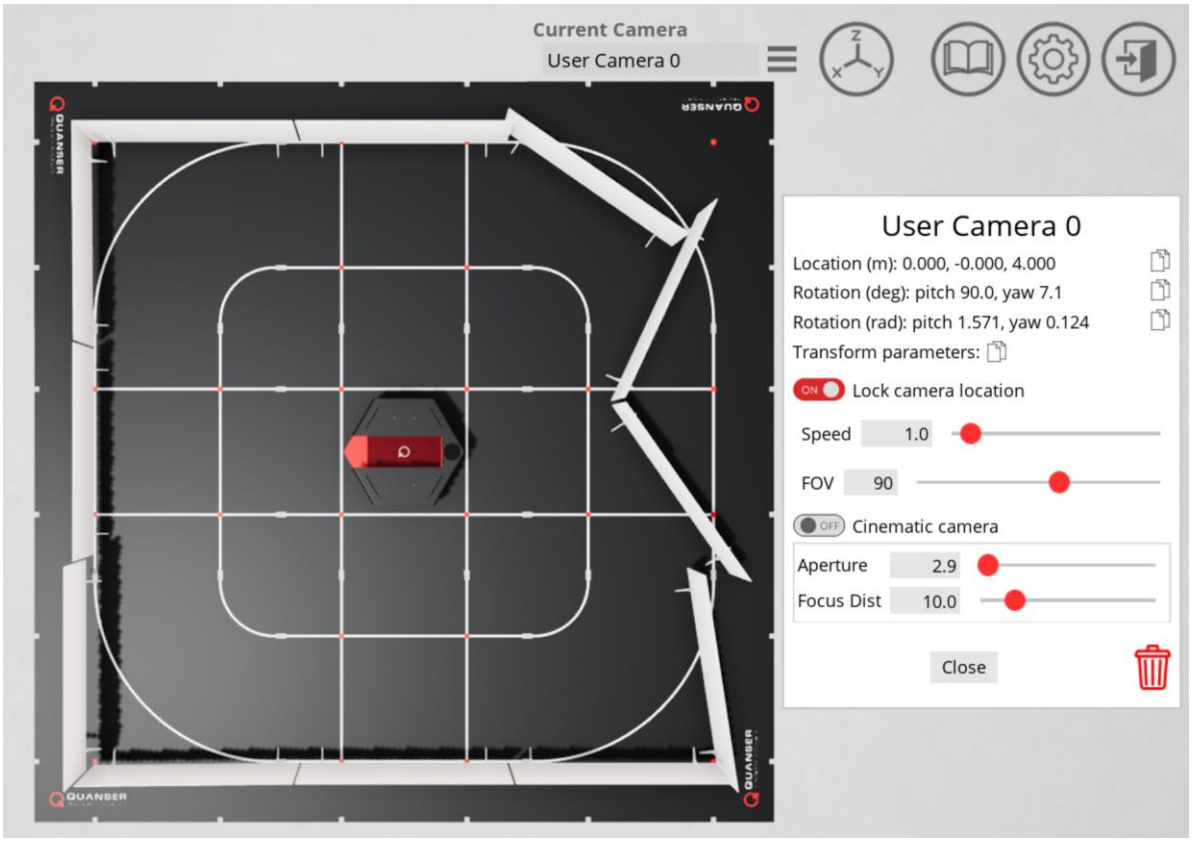

Figure 1. Successful set up of the Quanser Interactive Labs Workspace

Introduction to Scan Matching

1. Consider a situation, shown in Figure 2, where the QBot:

    a. Begins at location A
    b. Drives forward
    c. Stops at location B

    Suppose a LiDAR scan is taken at each location, giving you scan A and scan B. Describe how you would relate the data points in scan B to the points in scan A.

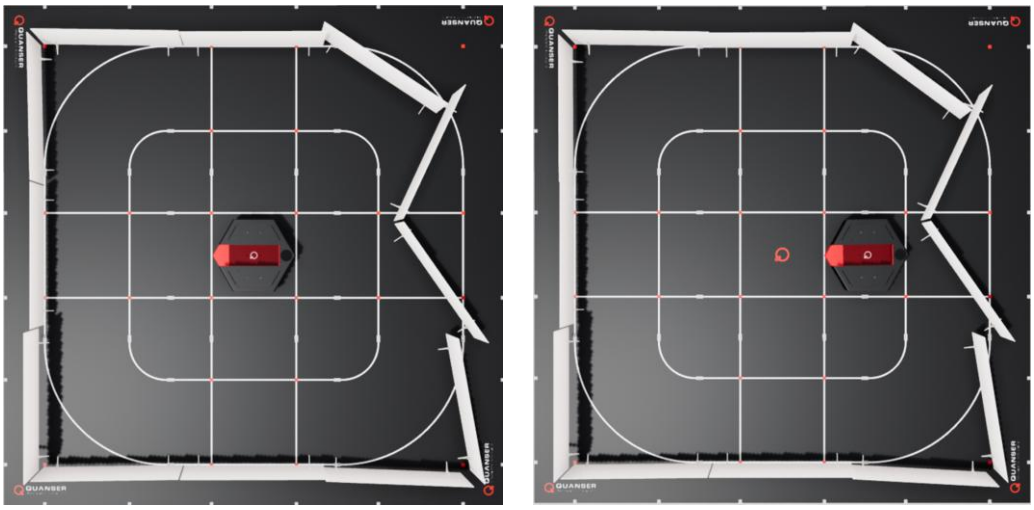

Figure 2. Top view of QBot in location A (left) and location B (right)

2. Consider a situation, shown in Figure 3, where the QBot:

    a. Begins facing direction C
    b. Rotates 90 degrees counterclockwise
    c. Stops rotating, so it is now facing direction D

    Suppose a LiDAR scan is taken at each location, giving you scan C and scan D. Describe how you would relate the data points in scan D to the points in scan C. What is the added challenge for rotation compared to translation?

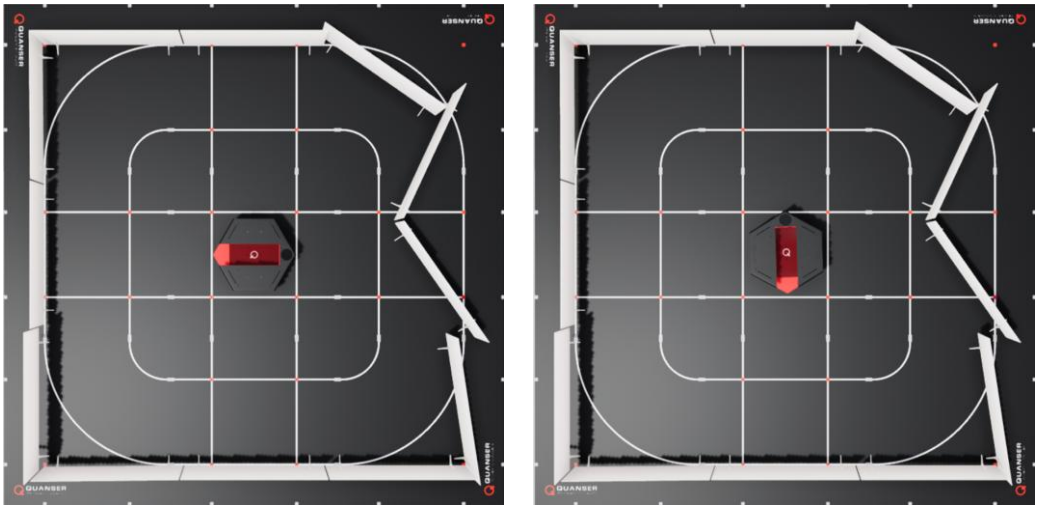

Figure 2. Top view of QBot facing direction C (left) and direction D (right)

Setting up LiDAR Scan Match

3. In this lab you will be exploring LiDAR-based localization using the Scan Match algorithm. Open localization.py. In section B, right click over **`QBPRanging()`** and select "go to definition" to view the class definition within the qbot_platform_functions module. Ensure that the functions **`adjust_and_subsample()`** and **`correct_lidar()`** have been completed according to the instructions in the **Obstacle Detection** lab.

4. In localization.py, fill in Section C to call **`adjust_and_subsample()`** and **`correct_lidar()`** following the **Obstacle Detection** lab if needed.

5. Next, we will be using the Scan Match function implemented in Python to estimate the position of the QBot using LiDAR scan data. Find **`QBPLocalization()`** in section B of localization.py and open the class definition.

6. Scroll down to find the **`scan_match()`** method, which calls the **`match()`** method from the **`Lidar2DMatchScansGrid`** class in Quanser's Image Processing API. The **`match()`** method matches a reference LiDAR scan to the current LiDAR scan, performing a grid matching algorithm to determine the translation and rotation of the current scan relative to the reference. To view descriptions of the method and its inputs, right-click on **`match()`** and click "go to definition".

7. In this lab, we will find the position of the QBot relative to its initial position. To do this, we save a copy of the LiDAR scan in its initial position as the reference scan. Complete the function **`saveRef()`** to save the current scan data. In localization.py, modify the if statement that calls this method in Section D and sets the flag **`refCollected`** once the reference scan has been saved.

8. Return to localization.py. Instead of collecting the reference scan immediately upon entering the main code loop, the reference scan should be collected after a short window of time (a few seconds) for the program to start up. In section D, add code that checks this condition before saving the current LiDAR scan using **`saveRef().`**

9. Return to the code that defines the **`scan_match()`** method. Currently, the parameters passed to the **`match()`** method, **`ref_ranges`**, **`ref_angles`**, **`num_ref_points`**, **`ranges`**, **`angles`**, and **`num_points`**, are set to **`None`**. Modify the code to pass in the correct variables to these parameters.

10. The **`match()`** method starts the grid search at an initial position passed in as the vector **`[x position, y position, theta]`** to the parameter **`initial_pose.`** Complete the function **`scan_match()`** to use the previous estimated position as a starting point for the current search and save it in **`prevPose`**.

11. In Section D of localization.py, modify the line that calls **`scan_match()`** to pass in the current LiDAR scan. The **`transRange`** parameter specifies the extent of the translation search, in m, in the x and y directions. Set this parameter to **`(3.0,3.0)`** in localization.py. Note as well that the **`rotRange`** parameter sets the angular search range in radians and is set to a default of 2π.

12. To visualize the QBot's position in the reference scan, each point in the reference scan is converted to Cartesian coordinates and stored in **`refX`** and **`refY`**, which are then plotted along with the estimated position on the same XY scope. This code is already implemented within localization.py and qbot_platform_functions.py, do not modify it.

13. Save your changes in both files and run localization.py. Ensure that the "Lock camera location" toggle switch is set to the "on" position to fix the camera location. Once the initial setup has elapsed, the code should open four scope windows. The scan match estimates are displayed in the Estimated Pose vs Time and Estimated Pose XY scopes. The Score scope displays a non-normalized confidence value for the match. The Loop Execution Time scope displays the duration of the loop that runs each time data is read from the QBot. Arrange your windows so you can view these scopes and Quanser Interactive Labs at the same time.

14. To drive the QBot using your keyboard, press and hold the space bar to arm the robot. Keep the space bar pressed as you drive the QBot using the movement keys. To stop the QBot, disarm it by releasing the space bar. The movement keys are as follows:

    a. Press the "A" and "D" keys to turn the QBot body

    b. Press the "I" and "K" keys to drive the QBot forward and back

    Drive the QBot around while observing the Estimated Pose scopes. Which axis corresponds to the forward movement direction of the QBot?

15. Observe the Loop Execution Time scope as you drive the QBot around, noting the amount of variation in the execution time. Referring to the code, provide reasons for the variation that we see in the loop timing. Press the "U" key to stop the code.

Exploring Parameters of LiDAR Scan Match

16. In this section, we will explore how scan match parameters impact performance. We will use the score, control loop execution time, and estimated location to compare performance. The **`resolution`** parameter sets the number of grid cells per meter. The **`QBPLocalization()`** object is initialized with resolution set to 10. Change the resolution to 20 and run the code. Drive the QBot around while observing the scopes. How did increasing the resolution impact performance? If needed, increase the resolution by increments of 5 until you observe a significant effect. Stop your code.

17. With the resolution set to 10, modify the line of code that calls **`scan_match`** so that **`transRange`** is set to **`(1.0,1.0)`**. Run the code and drive the robot around while observing the scopes. How is the performance affected by changing the translation search range?

18. Stop the code when complete by pressing the "U" key. Ensure that you save a copy of your completed files for review later. Close Quanser Interactive Labs.